

KUBERNETES

Top 50 Kubernetes Interview Questions & Answers

Beginner & Intern-Level Preparation Guide

Detailed explanations with key technical terms

Covers Architecture • Pods • Deployments • Services • Storage
Scaling • Probes • kubectl • YAML • Troubleshooting

This guide contains 50 carefully chosen Kubernetes interview questions spanning every core topic. Each answer is written in plain language but keeps the important technical terms you are expected to use in an interview. Bolded terms are the keywords worth remembering.

1. Fundamentals & Architecture

Q1. What is Kubernetes and why is it used?

Answer: Kubernetes (often shortened to **K8s**) is an open-source **container orchestration platform** originally built by Google and now maintained by the CNCF. It automates the **deployment, scaling, networking, and lifecycle management** of containerized applications across a group of machines. Instead of manually starting containers and restarting them when they fail, you declare a *desired state* (e.g. "run 3 copies of my app") and Kubernetes continuously works to make the actual state match that desired state. It is used because it provides **self-healing, automated rollouts/rollbacks, horizontal scaling, service discovery, and load balancing** in a consistent way across on-prem and cloud environments.

Q2. What problems does Kubernetes solve that plain Docker does not?

Answer: Docker is great at **building and running individual containers** on a single host, but it does not manage them at scale. Kubernetes adds the orchestration layer on top: it **schedules containers across many nodes**, restarts failed containers (**self-healing**), performs **rolling updates and rollbacks** without downtime, provides **service discovery and load balancing**, handles **automatic scaling**, and manages **configuration and secrets**. In short, Docker answers "how do I run one container?" while Kubernetes answers "how do I reliably run thousands of containers across many machines?"

Q3. What is the difference between a container and a virtual machine (VM)?

Answer: A **VM** virtualizes hardware and runs a full **guest operating system** on top of a hypervisor, making it heavy (gigabytes) and slow to start. A **container** virtualizes the operating system instead — it shares the host kernel and packages only the application and its dependencies, making it **lightweight (megabytes), fast to start, and portable**. Containers provide process-level isolation using Linux features like **namespaces and cgroups**, whereas VMs provide stronger hardware-level isolation. Kubernetes orchestrates containers, not VMs.

Q4. Explain the overall architecture of a Kubernetes cluster.

Answer: A Kubernetes cluster has two logical parts. The **Control Plane** (the "brain") makes global decisions and manages the cluster state; it includes the **API Server, etcd, Scheduler, and Controller Manager**. The **Worker Nodes** (the "muscle") actually run the application workloads; each node runs the **kubelet, kube-proxy, and a container runtime**. Users interact with the cluster through the API Server (usually via `kubectl`), and the control plane ensures the worker nodes run the workloads in the declared desired state.

Q5. What is the role of the API Server in Kubernetes?

Answer: The **kube-apiserver** is the **front-end and central hub** of the control plane. Every component — users, `kubectl`, controllers, and nodes — communicates through it. It **exposes the Kubernetes REST API**, validates and processes requests, and is the **only component that reads from and writes to etcd**. It acts as the single source of truth gateway and enforces authentication, authorization, and admission control before any change is persisted.

Q6. What is etcd and why is it important?

Answer: **etcd** is a distributed, consistent **key-value store** that serves as the **backing store for all cluster data** — the entire desired and current state of the cluster (objects, configs, secrets) lives here. It is critical because if etcd is lost and not backed up, the cluster state is lost. Only the API Server talks to etcd directly. Because it holds the cluster's source of truth, etcd is typically run in an **odd-numbered, highly available cluster** (3 or 5 nodes) and backed up regularly.

Q7. What does the Scheduler do?

Answer: The **kube-scheduler** watches for newly created Pods that have **no node assigned** and decides which worker node each Pod should run on. It makes this decision based on **resource requirements (CPU/memory requests), node capacity, affinity/anti-affinity rules, taints and tolerations, and other constraints**. The scheduler only *selects* the node; the kubelet on that node is what actually starts the Pod.

Q8. What is the Controller Manager?

Answer: The **kube-controller-manager** runs the **control loops** that continuously watch the cluster's state and work to drive the **actual state toward the desired state**. It bundles several controllers, such as the **Node Controller** (handles node failures), **ReplicaSet/Deployment Controller** (maintains the right number of Pods), **Job Controller**, and **Endpoints Controller**. Each controller follows the pattern: observe current state, compare to desired state, take corrective action.

Q9. What components run on every worker node?

Answer: Each worker node runs three key components: the **kubelet**, an agent that talks to the API Server and ensures the containers described in Pod specs are running and healthy; **kube-proxy**, which maintains network rules so that Services can route traffic to the correct Pods; and a **container runtime** (such as containerd or CRI-O) that actually pulls images and runs containers.

Q10. What is the role of the kubelet?

Answer: The **kubelet** is the primary **node agent** that runs on every worker node. It registers the node with the API Server, receives **PodSpecs**, and ensures the described containers are **running and healthy** via the container runtime. It also runs **liveness/readiness probes** and reports node and Pod status back to the control plane. Note: the kubelet only manages containers created by Kubernetes, not arbitrary ones.

2. Pods

Q11. What is a Pod and why is it the smallest unit in Kubernetes?

Answer: A **Pod** is the smallest and simplest deployable unit in Kubernetes. It represents **one or more containers** that are tightly coupled and share the same **network namespace (same IP and port space) and storage volumes**. Kubernetes does not deploy containers directly — it always wraps them in Pods. Most Pods run a single container, but multiple containers in one Pod is useful when they must work together as a unit (e.g. a main app plus a logging helper).

Q12. What is a multi-container Pod and when would you use one?

Answer: A **multi-container Pod** runs more than one container that share the Pod's network and storage. A common pattern is the **sidecar**, where a helper container augments the main one — for example, a log shipper, a proxy, or a sync agent. They are used when containers are **tightly coupled** and need to share resources and lifecycle. The **init container** pattern is another use, running setup tasks to completion before the main containers start.

Q13. Why are Pods considered ephemeral, and what is the consequence?

Answer: Pods are **ephemeral** — they are designed to be disposable and are **not self-healing on their own**. When a Pod dies, it is not resurrected; a new Pod with a **new IP address** is created in its place (by a controller like a Deployment). The consequences are: (1) you should not rely on a Pod's IP being stable — use a **Service** for stable networking; and (2) data stored inside the Pod is lost on restart unless you use a **persistent Volume**.

Q14. What is the difference between a Pod and a container?

Answer: A **container** is a single running instance of an image (the actual application process), while a **Pod** is the Kubernetes wrapper around one or more containers. Kubernetes schedules and manages **Pods**, not individual containers. Containers inside the same Pod share the Pod's IP, network ports, and volumes, and are always scheduled together onto the same node.

Q15. What are init containers?

Answer: **Init containers** are special containers that run **before** the main application containers in a Pod and must **complete successfully** before the app containers start. They run **sequentially, one at a time**. They are used for **setup tasks** such as waiting for a dependent service to be ready, cloning a git repo, or setting permissions. If an init container fails, Kubernetes restarts the Pod until it succeeds.

3. Workloads & Controllers

Q16. What is a ReplicaSet?

Answer: A **ReplicaSet** ensures that a **specified number of identical Pod replicas** are running at any given time. If a Pod dies, the ReplicaSet creates a new one; if there are too many, it deletes extras. It uses **label selectors** to identify the Pods it manages. In practice you rarely create ReplicaSets directly — you use a **Deployment**, which manages ReplicaSets for you.

Q17. What is a Deployment and what does it provide over a ReplicaSet?

Answer: A **Deployment** is a higher-level controller that manages **ReplicaSets** and provides **declarative updates** for Pods. Beyond simply maintaining replica count, it adds **rolling updates** (gradually replacing old Pods with new ones for zero downtime), **rollbacks** to previous versions, and **revision history**. It is the most common way to run stateless applications in Kubernetes.

Q18. Explain how a rolling update works in a Deployment.

Answer: A **rolling update** gradually replaces old Pods with new ones so the application stays available throughout. When you update a Deployment's image, it creates a **new ReplicaSet** and incrementally scales it up while scaling the old one down, controlled by **maxSurge** (how many extra Pods can be created) and **maxUnavailable** (how many Pods can be down at once). This ensures **zero or minimal downtime**. If something goes wrong, you can `kubectl rollout undo` to roll back.

Q19. What is the difference between a Deployment and a StatefulSet?

Answer: A **Deployment** is for **stateless** applications — Pods are interchangeable, have random names, and any Pod can be replaced by any other. A **StatefulSet** is for **stateful** applications (like databases) that need **stable, unique network identities** (predictable names like app-0, app-1), **stable persistent storage per Pod**, and **ordered, graceful deployment and scaling**. Use a StatefulSet when each Pod's identity and data must be preserved across restarts.

Q20. What is a DaemonSet and when is it used?

Answer: A **DaemonSet** ensures that a copy of a specific Pod runs on **every node** (or a selected subset of nodes) in the cluster. When new nodes join, the Pod is automatically added; when nodes leave, the Pod is removed. It is used for **node-level background services** such as **log collectors** (Fluentd), **monitoring agents** (node exporters), and **networking/storage daemons**.

Q21. What is the difference between a Job and a CronJob?

Answer: A **Job** creates one or more Pods to run a **task to completion** (a batch task) and then stops — for example, a database migration or a one-off data processing run. A **CronJob** creates Jobs on a **repeating schedule** defined with cron syntax (e.g. "every night at 2 AM") — for example, periodic backups or report generation. In short, a Job runs once; a CronJob runs Jobs on a schedule.

4. Networking & Services

Q22. What is a Service in Kubernetes and why is it needed?

Answer: A **Service** is an abstraction that provides a **stable network endpoint (a fixed virtual IP and DNS name)** for a logical set of Pods, selected via **labels**. It is needed because Pods are ephemeral and get new IPs when recreated — a Service decouples clients from individual Pod IPs and also provides **load balancing** across the matching Pods. Clients talk to the Service, and the Service forwards traffic to healthy backing Pods.

Q23. Explain the different Service types: ClusterIP, NodePort, and LoadBalancer.

Answer: **ClusterIP** (the default) exposes the Service on an **internal cluster IP**, reachable only from within the cluster. **NodePort** exposes the Service on a **static port on every node's IP**, allowing external access via `NodeIP:NodePort`. **LoadBalancer** provisions an **external cloud load balancer** (on supported clouds) that routes external traffic to the Service. Each type builds on the previous: LoadBalancer includes a NodePort, which includes a ClusterIP.

Q24. What is Ingress and how does it differ from a Service?

Answer: **Ingress** is an API object that manages **external HTTP/HTTPS access** to Services, providing **host- and path-based routing**, **TLS/SSL termination**, and a single entry point for many Services. It requires an **Ingress Controller** (like NGINX or Traefik) to actually do the routing. The difference: a **Service** (e.g. LoadBalancer) exposes one service, often needing one load balancer each, while **Ingress** lets you route to many services through one load balancer using rules — it's a smart Layer 7 router.

Q25. How does service discovery and DNS work in Kubernetes?

Answer: Kubernetes runs an internal **DNS service** (CoreDNS) that automatically gives every Service a **DNS name**. Pods can reach a Service by name — for example `my-service.my-namespace.svc.cluster.local`, or just `my-service` within the same namespace. This is **service discovery**: applications don't need hardcoded IPs; they use stable DNS names, and the DNS resolves to the Service's ClusterIP, which then load-balances to the Pods.

Q26. What does kube-proxy do?

Answer: **kube-proxy** runs on each node and maintains the **network rules** (using iptables or IPVS) that implement Kubernetes Services. When traffic is sent to a Service's virtual IP, kube-proxy ensures it is **routed and load-balanced to one of the backing Pods**. It effectively makes the Service abstraction work at the network level on each node.

5. Configuration & Storage

Q27. What is a ConfigMap and why use one?

Answer: A **ConfigMap** stores **non-sensitive configuration data** as key-value pairs, separate from the container image. You use it to **externalize configuration** — like environment settings, URLs, or config files — so the same image can be reused across environments without rebuilding. ConfigMap data can be injected into Pods as **environment variables** or **mounted as files**.

Q28. What is a Secret and how does it differ from a ConfigMap?

Answer: A **Secret** stores **sensitive data** such as passwords, API keys, tokens, and TLS certificates. It works like a ConfigMap (injected as env vars or mounted files) but is intended for confidential data: values are **base64-encoded**, can be **encrypted at rest** in etcd, and access can be tightly controlled with RBAC. The key difference is purpose: ConfigMaps are for **non-sensitive** config; Secrets are for **sensitive** data. (Note: base64 is encoding, not encryption, so RBAC and encryption-at-rest matter.)

Q29. What is a Volume in Kubernetes?

Answer: A **Volume** is a directory accessible to the containers in a Pod, used to **persist or share data** beyond a single container's lifetime. Unlike the container's own filesystem (which is wiped on restart), a Volume's lifetime is tied to the **Pod** (for most types). Volumes also let multiple containers in a Pod share files. Types range from temporary (`emptyDir`) to persistent cloud/network storage.

Q30. Explain PersistentVolume (PV) and PersistentVolumeClaim (PVC).

Answer: A **PersistentVolume (PV)** is a piece of **actual storage in the cluster** (e.g. a cloud disk or NFS share), provisioned by an admin or dynamically. A **PersistentVolumeClaim (PVC)** is a **user's request for storage** with a certain size and access mode. Kubernetes **binds** a PVC to a suitable PV. This separation lets developers ask for storage (PVC) without knowing the underlying storage details (PV) — similar to how Pods consume compute without choosing a specific node.

6. Organization (Namespaces, Labels)

Q31. What are Namespaces and why are they useful?

Answer: **Namespaces** provide a way to **logically divide a single cluster** into multiple virtual sub-clusters. They are useful for **isolating environments or teams** (e.g. dev, test, prod), **avoiding name collisions** (same resource name can exist in different namespaces), and applying **resource quotas and access controls (RBAC)** per namespace. Default namespaces include `default`, `kube-system`, and `kube-public`.

Q32. What are Labels and Selectors?

Answer: **Labels** are key-value pairs attached to objects (like Pods) for **identification and grouping** — e.g. `app=frontend`, `env=prod`. **Selectors** are queries used to **filter and select** objects by their labels. This is the core mechanism Kubernetes uses to connect things: a Service uses a selector to find its Pods, and a ReplicaSet uses one to track the Pods it manages. Labels are meaningful to users; they organize and link resources.

Q33. What is the difference between labels and annotations?

Answer: **Labels** are **identifying metadata** used by selectors to group and select objects (they are queryable). **Annotations** are **non-identifying metadata** used to attach arbitrary, often larger, information for tools and humans — like build versions, contact info, or config used by external systems. The rule of thumb: if you need to **select/filter by it**, use a label; if it's just **extra descriptive info**, use an annotation.

7. Health, Scaling & Reliability

Q34. What is the difference between a liveness probe and a readiness probe?

Answer: A **liveness probe** checks whether a container is **still alive/healthy**; if it fails, Kubernetes **restarts the container** (used to recover from deadlocks/hangs). A **readiness probe** checks whether a container is **ready to serve traffic**; if it fails, the Pod is **removed from Service endpoints** (no traffic is sent) but is **not restarted**. In short: liveness controls **restarts**, readiness controls **traffic**.

Q35. What is a startup probe and when is it used?

Answer: A **startup probe** checks whether an application has **finished starting up**. It is used for **slow-starting containers**: while the startup probe is running, the liveness and readiness probes are disabled so a slow boot isn't mistaken for a failure and killed prematurely. Once the startup probe succeeds, the regular liveness and readiness probes take over.

Q36. How does Kubernetes achieve self-healing?

Answer: Kubernetes self-heals through **controllers and control loops** that constantly compare actual state to desired state. If a Pod crashes, the **ReplicaSet/Deployment** recreates it to maintain replica count. If a node fails, the **Node Controller** notices and Pods are rescheduled onto healthy nodes. **Liveness probes** restart unhealthy containers. The system continually drives the cluster back to the declared desired state without manual intervention.

Q37. What is the Horizontal Pod Autoscaler (HPA)?

Answer: The **Horizontal Pod Autoscaler** automatically **scales the number of Pod replicas** in a Deployment/ReplicaSet up or down based on observed **metrics** such as CPU or memory utilization (or custom metrics). For example, it can keep average CPU at 50% by adding Pods under load and removing them when idle. "Horizontal" means **adding/removing Pods**, as opposed to vertical scaling (changing a Pod's CPU/memory).

Q38. What are resource requests and limits?

Answer: **Requests** specify the **minimum CPU/memory a container is guaranteed**, and the scheduler uses them to decide which node has room. **Limits** specify the **maximum a container may use**. If a container exceeds its **memory limit**, it is **terminated (OOMKilled)**; if it exceeds its **CPU limit**, it is **throttled**. Setting both prevents a single Pod from starving others and helps the scheduler pack workloads efficiently.

Q39. How do you roll back a bad deployment?

Answer: Kubernetes keeps a **revision history** of a Deployment. To undo a problematic update, run `kubectl rollout undo deployment/<name>`, which reverts to the **previous ReplicaSet/revision**. You can target a specific revision with `--to-revision`, and inspect history with `kubectl rollout history`. Because Deployments track revisions, rollbacks are fast and safe.

8. kubectl & YAML Manifests

Q40. What is kubectl?

Answer: **kubectl** is the official **command-line tool** for interacting with a Kubernetes cluster. It sends commands to the **API Server** to create, view, update, and delete resources. Common verbs include `get`, `describe`, `apply`, `delete`, `logs`, `exec`, and `scale`. It reads cluster connection details from a **kubeconfig** file.

Q41. What is the difference between 'kubectl apply' and 'kubectl create'?

Answer: `kubectl create` is **imperative** — it creates a resource and errors if it already exists. `kubectl apply` is **declarative** — it creates the resource if missing or **updates it to match the file** if it already exists, tracking changes so you can repeatedly apply the same manifest. For managing resources from version-controlled YAML, `apply` is preferred.

Q42. What are the four required top-level fields in a Kubernetes YAML manifest?

Answer: Every manifest needs: **apiVersion** (which API version/group the object belongs to, e.g. `apps/v1`); **kind** (the object type, e.g. `Deployment`); **metadata** (identifying info like `name`, `namespace`, and `labels`); and **spec** (the desired state/configuration of the object). The control plane reads `spec` and works to make the actual state match it.

Q43. Which kubectl command would you use to debug a failing Pod, and what does it show?

Answer: Use `kubectl describe pod <name>` to see detailed information including **events** (scheduling failures, image pull errors, probe failures), container statuses, and restart counts. Combine it with `kubectl logs <name>` to view application output and `kubectl get pods` to check status. For live debugging, `kubectl exec -it <name> -- /bin/sh` opens a shell inside the container.

Q44. What is the difference between declarative and imperative approaches in Kubernetes?

Answer: The **imperative** approach issues explicit commands describing *how* to reach a state (e.g. `kubectl run`, `kubectl create`) — good for quick, one-off actions. The **declarative** approach defines the *desired end state* in YAML manifests and lets Kubernetes figure out how to get there (e.g. `kubectl apply -f`) — preferred for production because it's **repeatable**, **version-controllable**, and **self-documenting**.

9. Troubleshooting

Q45. What does CrashLoopBackOff mean and how do you investigate it?

Answer: **CrashLoopBackOff** means a container **starts, crashes, and Kubernetes keeps restarting it**, with an increasing back-off delay between attempts. Common causes: an application error/misconfiguration, a missing dependency or env var, a failing liveness probe, or an incorrect start command. To investigate, check `kubectl logs <pod>` (and `--previous` for the crashed instance) and `kubectl describe pod <pod>` for events.

Q46. What does ImagePullBackOff mean?

Answer: **ImagePullBackOff** means Kubernetes **cannot pull the container image**, so it backs off and retries. Common causes: a **wrong image name or tag**, the image doesn't exist, the **registry is private and credentials (imagePullSecrets) are missing or wrong**, or network/registry connectivity issues. Inspect with `kubectl describe pod <pod>` to see the exact pull error in the events.

Q47. A Pod is stuck in 'Pending' state. What are the likely causes?

Answer: A **Pending** Pod has been accepted but **not yet scheduled to a node**. Likely causes: **insufficient resources** (no node has enough CPU/memory to satisfy the Pod's requests), **unsatisfiable scheduling constraints** (node selectors, affinity, taints/tolerations), or an **unbound PersistentVolumeClaim** (no matching storage). Run `kubectl describe pod <pod>` and read the scheduler events to find the exact reason.

Q48. What is the difference between horizontal and vertical scaling?

Answer: **Horizontal scaling** (scaling out/in) means **adding or removing Pod replicas** to handle load — this is the typical Kubernetes approach, often automated with the HPA. **Vertical scaling** (scaling up/down) means **increasing or decreasing the CPU/memory of existing Pods**. Horizontal scaling is generally preferred because it improves availability and avoids per-Pod resource ceilings; vertical scaling can require restarting Pods.

10. Core Concepts (Misc)

Q49. What is a Namespace's role in RBAC and resource quotas?

Answer: Namespaces act as a **scope boundary** for both access control and resource governance. With **RBAC (Role-Based Access Control)**, you can grant users permissions limited to a specific namespace (via Roles and RoleBindings). With **ResourceQuotas** and **LimitRanges**, you can cap the total CPU/memory/object count a namespace may consume. Together they let multiple teams safely share one cluster.

Q50. What is the desired state model and why is it central to Kubernetes?

Answer: The **desired state model** is the core philosophy of Kubernetes: you **declare what you want** (e.g. "3 replicas of this image, exposed on port 80") in manifests, and Kubernetes' **controllers continuously reconcile** the actual cluster state to match. You don't script the steps; you describe the goal. This makes the system **self-healing, predictable, and idempotent** — applying the same manifest repeatedly always converges to the same state, which is the foundation of declarative infrastructure.
